PNI ソフトウェア健全性診断レポート

本レポートは、Composer・フレームワーク不使用の手続き型レガシー素PHP EC・管理アプリケーション `legacy_shop_messy` (`v0.0.0` , `raw_php`) に対して、PNI ステートフル監査ワークフロー `php_legacy_shop_messy` （3観点アーキテクチャ記録・PHP AST 静的解析・隔離 Docker コンテナによるビルド/構文/起動スモーク検証・Trivy 脆弱性/IaC/Secret スキャン・ `.pni/pentests/` 隔離ループバック動的 Web ペネトレーションテスト）を実行した包括的診断結果である。



legacy_shop_messy (raw_php) PNI 総合健全性スコアカード

診断メタデータ概要

- **対象プロジェクト**: `legacy_shop_messy` (`v0.0.0` / 対象ルート: `examples/php/legacy_shop_messy`)
- **対象言語・ビルド基盤**: `php` (`raw_php` ：Composer 未導入、 `Dockerfile` は EOL の `php:7.4-apache` + `ext-mysqli` 、本番6ファイル・計 `107` LOC)
- **診断ワークフロー / Run ID**: `php_legacy_shop_messy` / `wf-1abc5e13`
- **評価実施日時**: `2026-10-24T15:02:00.000000+00:00`
- **総合スコア・グレード**: **15 / 100 (Grade D)**

- **エンジニアリングルール適合率**: `0 / 17` ルール PASS（`FAIL`: 15件、`NOT_APPLICABLE`: 2件［Laravel / WordPress］）
- **動的Webペネトレーションテスト (`web_pentest`)**: `FAILED`（全`19`プローブ中`9` PASS / `10` FAIL：`CRITICAL: 2`［無認証RCE `PNI_RCE_CANARY_8842`・マジックハッシュ `'240610708'` 管理者突破］、`HIGH: 4`、`MEDIUM: 2`、`LOW: 2`）

Table of Contents

1. エグゼクティブサマリー

2. アーキテクチャとソースコード構成

3. 推奨する改善ロードマップ

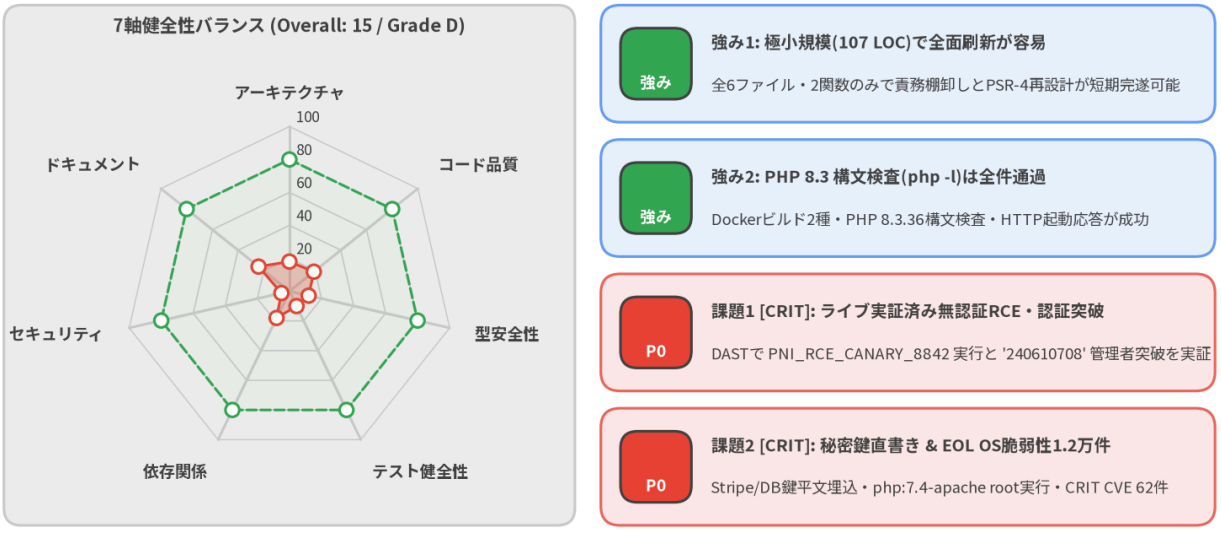
4. 評価指標とルール適合マトリクス

5. コード品質・型安全性・ランタイム検証詳細

6. セキュリティ・脆弱性・サプライチェーン分析

1. エグゼクティブサマリー

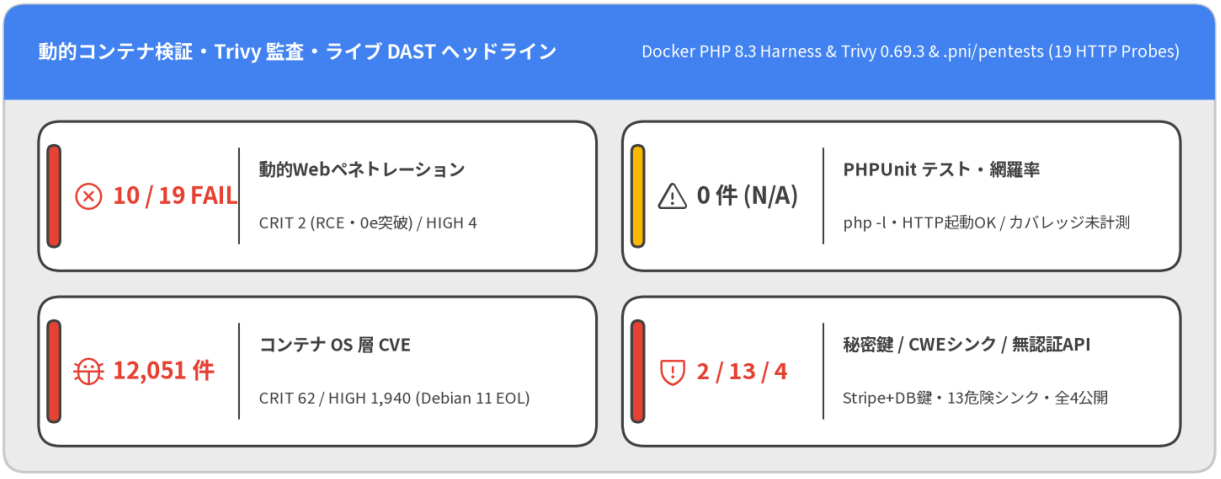
レガシー素PHP EC・管理アプリケーション `legacy_shop_messy` の総合健全性スコアは **15/100 (Grade D)** と診断された。`composer.json` / `composer.lock`・PSR-4 オートローディング・`declare(strict_types=1)`・自動テストスイートがいずれも未導入（`0%`）である上、隔離ループバック DAST（`web_pentest`：全 `19` プローブ中 `9` PASS / `10` FAIL）により、無認証 `POST /admin_backup.php` における OS コマンド実行（`PNI_RCE_CANARY_8842` 出力、`CWE-78` / `CWE-306` CRITICAL）と `POST /login.php` の MD5 マジックハッシュ `'240610708'` 管理者認証突破（`'Logged in as admin'`、`CWE-697` CRITICAL）、`/index.php` の LFI トラバーサル露出（`CWE-22` HIGH）、`/index.php`・`/product.php` の反射型 XSS（`CWE-79` HIGH×2）が実 HTTP 通信で完全実証された。さらに SQLi 3箇所（`CWE-89`）、Cookie `unserialize`（`CWE-502`）、`includes/db.php` の本番 DB パスワード・Stripe Live 秘密鍵直書き（`CWE-798`）、EOL `php:7.4-apache` root コンテナ上の **12,051件の OS 層脆弱性（CRITICAL 62件、HIGH 1,940件）** が複合しており、即時封じ込めと PHP 8.3 モダン構成への刷新が必須である。



7軸エンジニアリング健全性レーダーと強み・最優先リスク要約

7軸評価サマリーと主要診断結論

- **全7軸が25点未満に沈む重大リスクプロファイル**: セキュリティ（`5`）・テスト健全性（`10`）・型安全性（`12`）が特に低く、アーキテクチャ（`18`）・依存関係（`18`）・コード品質（`19`）・ドキュメント（`24`）も含め、推奨基準（80点）との乖離が極めて大きい。全8監査ディメンションのすべてが Grade D（`web_penetration_audit` および `php_test_audit` は `0` 点）、適用可能15ルールのすべてが `FAIL`（`0/17` PASS）となった。
- **ライブ実証された RCE / 認証バイパスとコンテナ root 権限の危険な結合**: `.pni/pentests/examples/php/legacy_shop_messy/` による実 HTTP プローブで、`POST /admin_backup.php` の無認証シェル実行（`PNI_RCE_CANARY_8842`）と `POST /login.php` の `'240610708'` マジックハッシュ突破が実際に成立した。これが `Dockerfile` の `root` 実行（Trivy `DS-0002`）および `includes/db.php` の平文 DB パスワード・Stripe Live 秘密鍵と直結しており、単一リクエストで完全掌握に至る。



コンテナ動的検証・Trivyスキャン・ライブWebペネトレーションの主要指標

エグゼクティブ結論

- **強み (Verified Strengths)**: 総コード量が6ファイル・`107` LOC（関数2個）と極めてコンパクトであり、隔離コンテナ内での PHP 8.3.36 構文チェック（`php -l`、`1.01s`）および HTTP 起動応答（`1.06s`）も通過した。またサードパーティの脆弱な Composer パッケー

ジ負債も抱えていないため（`app_vuln_counts=0`）、短期集中での全面モダナイゼーションが現実的である。

- **最優先アクション (P0 Priorities)**: (1) ライブ DAST で悪用実証された `admin_backup.php` の無認証 `shell_exec` 廃止・認可強制、`login.php` の `extract()` / `0e` マジックハッシュ撤廃と `password_verify()` 化、`index.php` の `unserialize` / 動的 `include` 撤廃、全 SQL の PDO プリペアド化、全 HTML 出力エスケープと `.pni/pentests/` 回帰ゲート化、(2) `includes/db.php` の DB パスワード・Stripe Live 秘密鍵の即時失効ローテーションと環境変数化、(3) `Dockerfile` の `php:8.3-apache-bookworm` + 非 root `USER www-data` + `.dockerignore` への刷新を最優先で断行すべきである。

2. アーキテクチャとソースコード構成

`legacy_shop_messy` はクラスや名前空間を一切持たない **手続き型素PHPページスクリプト構造（本番6ファイル・計107 LOC + `Dockerfile` + `README.md`）** である。本章では、(1) ディレクトリ構造と HTML+SQL 混在スパゲッティ・相互 `include_once` 循環、(2) 無認証エンドポイントからの CWE 攻撃データフロー、(3) EOL `php:7.4-apache` root コンテナ実行トポロジの3観点からシステム構造を可視化する。

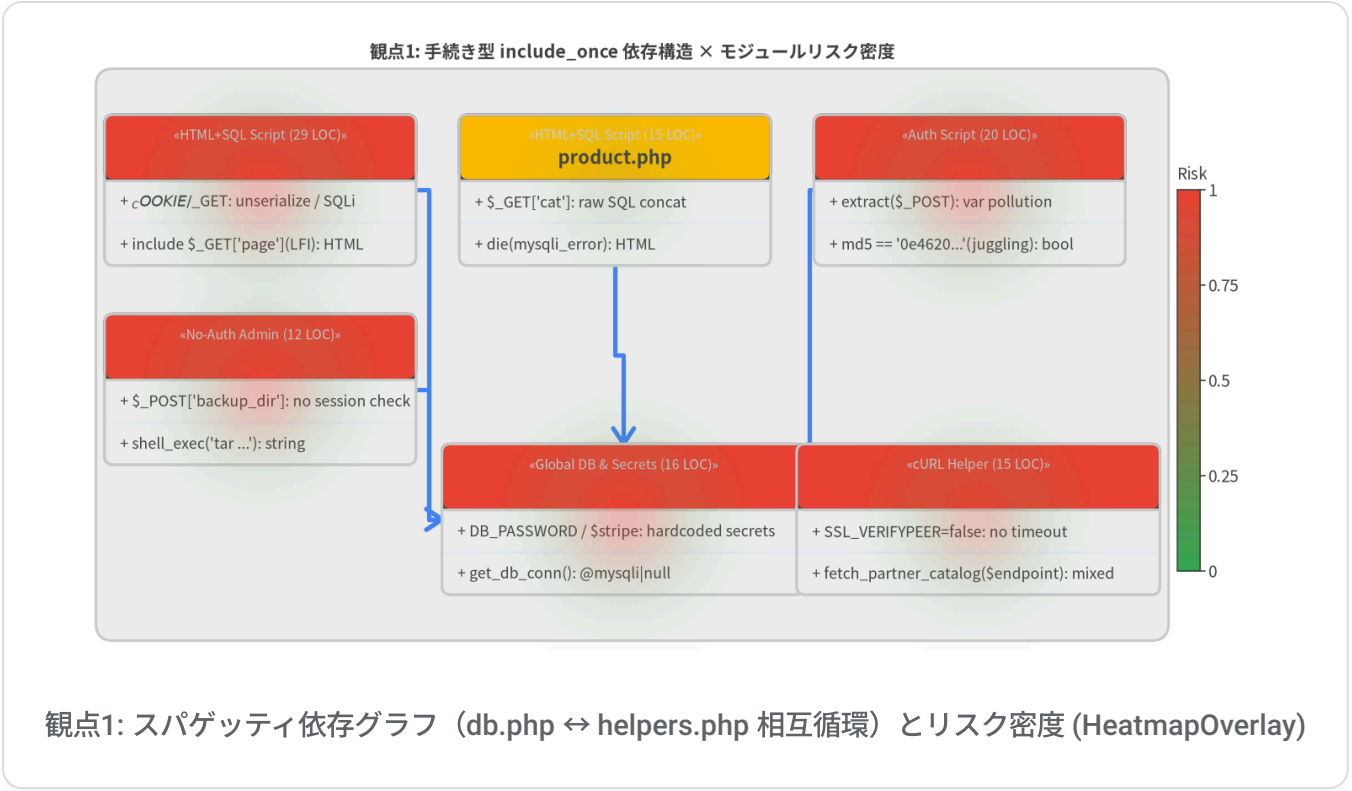


ファイルパス	LOC	関数・公開責務と主要アーキテクチャ/保安上の問題点
index.php	29	関数なし | 商品検索・カート復元・動的ページ読込 (HTML+SQL混在) [Risk 1.00] Cookie unserialize・SQLi・DAST実証済XSS/LFI
login.php	20	関数なし | POST管理者認証・session_start() [Risk 1.00] extract($_POST)・SQLi・DAST実証 '240610708' 突破
product.php	15	関数なし | カテゴリ商品取得・プロモーション表示 (HTML+SQL混在) [Risk 0.28] $_GET['cat'] SQLi・mysqli_error露出・DAST実証XSS
admin_backup.php	12	関数なし | アップロードディレクトリのtar圧縮管理画面 [Risk 1.00] 認可なし・shell_exec(tar) DAST実証済無認証RCE
includes/db.php	16	get_db_conn() | MySQL接続初期化・認証定数定義 [Risk 1.00] helpers.php相互循環・DB/Stripe鍵直書き・@抑制
includes/helpers.php	15	fetch_partner_catalog($endpoint) | 外部カタログcURL取得 [Risk 0.86] db.php相互循環・SSL検証無効・Timeout欠如・var_dump
Dockerfile / README.md	6 / 4	FROM php:7.4-apache (root実行・OS CVE 12,051件・TRACE/Header欠落) README は実質2行のみ (/var/www/html コピー手順1文)

ソースディレクトリ構造ツリーとファイル構成・責務一覧 (全6 PHPファイル・107 LOC)

観点1：HTML+SQL 混在スパゲッティ構造と相互 `include_once` 循環・リスク密度

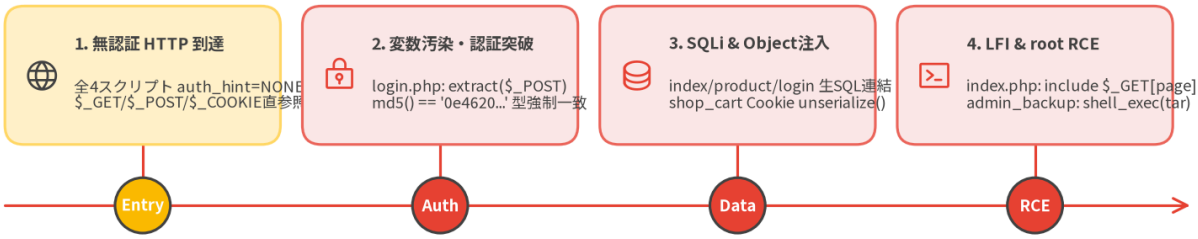
全4エントリスクリプト（`index.php`, `product.php`, `login.php`, `admin_backup.php`）は手動で `include_once 'includes/db.php';` を呼び出し、単一ファイル内でスーパーグローバル参照・生 SQL クエリ（または `shell_exec`）・インライン HTML 描画を実行している。さらに **`includes/db.php:2` が `includes/helpers.php` を読み込み、`includes/helpers.php:2` が逆に `includes/db.php` を読み込む双方向 `include_once` 循環** が形成されており、関心の分離と単方向レイヤ原則が完全に崩れている。



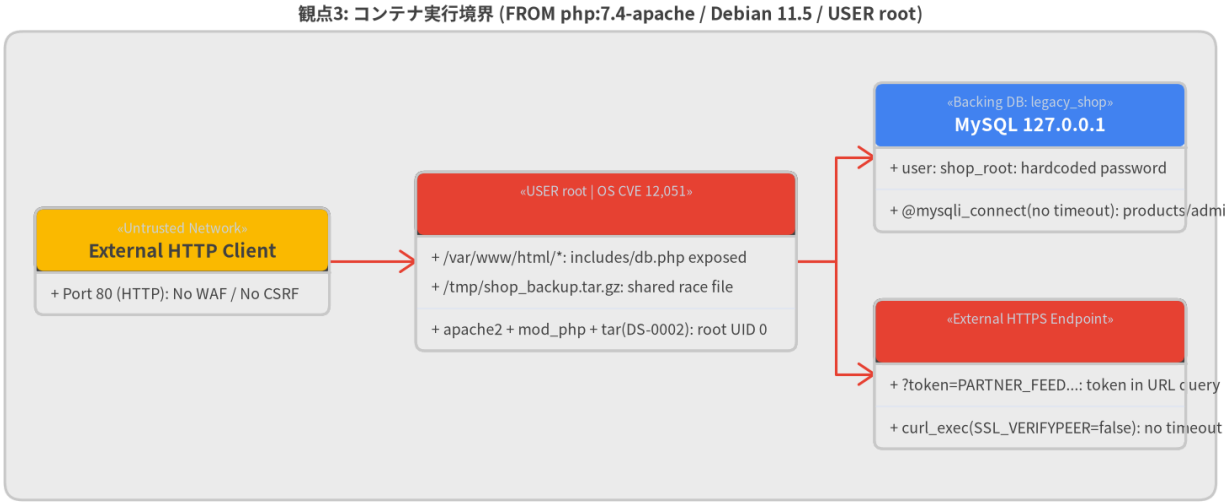
観点1: スパゲッティ依存グラフ（db.php ↔ helpers.php 相互循環）とリスク密度 (HeatmapOverlay)

観点2・観点3：無認証エンドポイント攻撃データフローと EOL `php:7.4-apache` コンテナ実行トポロジ

公開ドキュメントルート `/var/www/html/` 直下にすべての `.php` ファイルが配置されており（`Dockerfile:4 COPY . /var/www/html/`）、外部からの未検証 HTTP リクエストが認可ガードや型検証を経由せず危険なシンクへ直結している。加えて Apache ワーカーが `root` 権限で動作し、固定ファイル `/tmp/shop_backup.tar.gz` への排他なし書き込みやタイムアウト未設定の外部通信を行うため、セキュリティ侵害とサービス停止の双方に対して無防備な実行トポロジとなっている。



観点2: 無認証 .php エンドポイントから危険シンクへの攻撃データフロー



観点3: EOL php:7.4-apache root コンテナ実行トポロジとバッキングサービス接続境界

3. 推奨する改善ロードマップ

検出された計33件の指摘事項（`CRITICAL` 10件、`HIGH` 12件、`MEDIUM` 9件、`LOW` 2件）を体系的に解消し、総合スコアを Grade D (`15`) から本番運用可能な Grade A 水準（`85+`）へ引き上げるため、PNI が優先度順に正規化した **Phase 1（P0 緊急脆弱性封じ込め・DAST実証経路遮断・秘密鍵失効・PHP 8.3 非 root コンテナ刷新：4重点施策） -> Phase 2（P1 Composer PSR-4 レイヤ分離・PDO/password_hash・12-Factor/専用Health 整備：4重点施策） -> Phase 3（P2 `strict_types=1`・PHPUnit 自動テスト・OpenSSF ガバナンス確立：3重点施策）** の3段階ロードマップを策定する。



優先度フェーズ別 改善ロードマップ (Phase 1 [P0] -> Phase 2 [P1] -> Phase 3 [P2])

フェーズ別実行計画とトレーサビリティ

- **Phase 1 (`P0` ー 即時対応：ライブ実証脆弱性の緊急遮断・秘密情報ローテーション・EOL コンテナ廃止)**:
    1. `Dockerfile` を `php:7.4-apache` から現役サポートの `php:8.3-apache-bookworm` + `USER www-data` + `.dockerignore` + `HEALTHCHECK` へ刷新し、OS 層 CVE `12,051` 件（`CRITICAL` 62件、`HIGH` 1,940件）と `DS-0002`（root 実行）を一括解消する。
    2. `includes/db.php` に直書きされた MySQL パスワードと Stripe Live 秘密鍵を即時失効（ローテーション）し、環境変数 / Secret Manager 注入へ切り替える。

3. ライブ DAST で悪用実証された `POST /admin_backup.php` の無認証 `shell_exec`（`PNI_RCE_CANARY_8842`）と `POST /login.php` の `'240610708'` マジックハッシュ突破、`index.php` の LFI・反射型 XSS、`product.php` の `promo_html` XSS、HTTP セキュリティヘッダ欠落を緊急修正し、`.pni/pentests/examples/php/legacy_shop_messy/` 全19プローブの PASS を CI リリースゲート化する。
4. 残る SAST 危険シンク（`index.php:4` の `unserialize($_COOKIE)`、全3箇所の生 SQL 連結、`includes/helpers.php:9` の `CURLOPT_SSL_VERIFYPEER=false`）を PDO プリペアド化・署名付き JSON / サーバセッション化・TLS 検証強制で全面封鎖する。

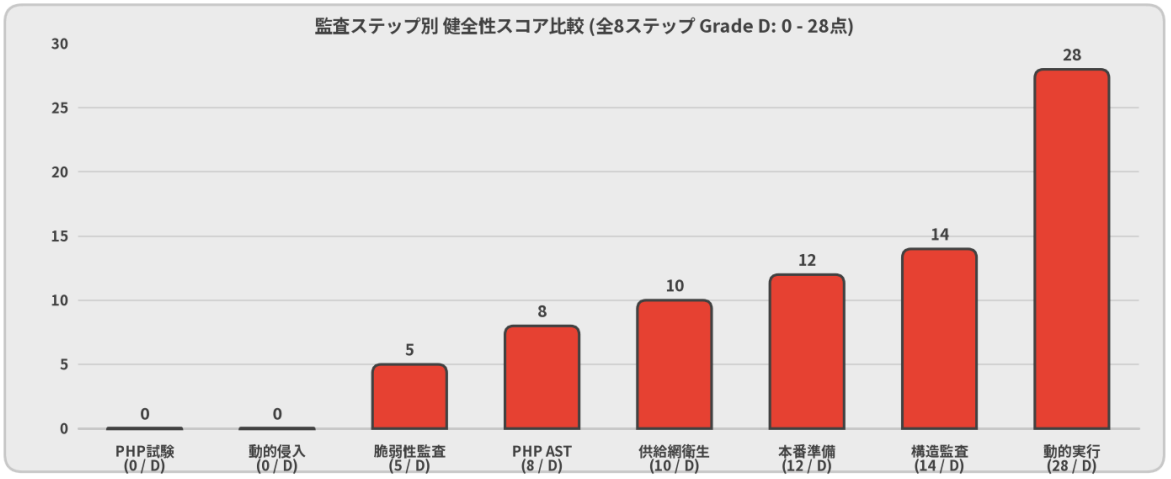
- **Phase 2 (`P1` ー 短期優先：Composer PSR-4 レイヤ分離・12-Factor 設定・専用 `/healthz` 整備)**: `composer.json` / `composer.lock` を導入して `public/index.php` 単一フロントコントローラ + `src/`（`Controller -> Service -> Repository`）の単方向 PSR-4 構造へ再編し、`includes/` の相互 `include_once` 循環とドキュメントルート露出を解消する。併せて `AppConfig` による環境変数集約、cURL / DB タイムアウト設定、Monolog PSR-3 構造化ログ、専用の `/healthz`・`/readyz` JSON エンドポイントを実装する。
- **Phase 3 (`P2` ー 中期定着：`strict_types=1`・PHPUnit 自動テスト・OpenSSF ガバナンス)**: 全 PHP ファイルへの `declare(strict_types=1);` とネイティブ型宣言・PHPStan (level 8) の適用、インメモリ `pdo_sqlite` を用いたオフライン PHPUnit 11 + `pcov`（カバレッジ 80% 以上）の整備、および `.gitignore`・`LICENSE`・`SECURITY.md`・最小権限 GitHub Actions CI + Dependabot の導入を完了する。

優先度	施策タイトル	対象ファイル	期待効果 (Impact)	関連ルール / CVE
P0	PHP 8.3 非rootセキュアコンテナへの即時刷新	Dockerfile	OS CVE 12,051件(CRIT 62)とDS-0002 root実行を一括解消	security_vulnerability CVE-2023-25690 / DS-0002
P0	DBパスワード・Stripe本番秘密鍵の失効と環境変数化	includes/db.php	平文認証情報の漏洩リスク(CWE-798)を完全に根絶	common/security SEC-03
P0	ライブ実証RCE・0e突破・LFI・XSS遮断とDAST回帰ゲート化	admin_backup / login / index / product / Docker	実HTTP再現した全10件の動的脆弱性根絶とCI 19プローブ監視	security_vulnerability SEC-06 (.pni/pentests)
P0	CMDi・LFI・unserialize・SQLi・TLS無効化の全面封鎖	admin_backup / index / product / login / helpers	無認証RCE・Object注入・SQLi・MITMの全13 CWEシンク遮断	security_vulnerability php/modern_stack
P1	HTML+SQL分離と相互include循環解消 (PSR-4レイヤ化)	index / product / login includes/db, helpers	Controller/Service/Repository分離とグローバル状態結合解消	common/architecture php/code_quality
P1	12-Factor設定集約・I/O Timeout・PSR-3ログ導入	includes/db, helpers product / admin_backup	cURLハングによるワーカー枯渇防止とJSON構造化ログ確立	common/twelve_factor production_readiness
P1	専用 /healthz・/readyz 実装と Dockerfile HEALTHCHECK	index.php / Dockerfile includes/db.php	Apache本番環境での確実な起動ヘルス検証と自動復旧確立	runtime_verification production_readiness
P1	Composer (composer.lock) 導入と公開ルート隔離	Dockerfile includes/db, helpers	再現可能な依存固定と公開Webルートからの内部コード隔離	php/dependencies common/supply_chain
P2	OpenSSFガバナンス文書と最小権限CI・Dependabot整備	Dockerfile README.md	.gitignore/LICENSE/SECURITY整備と継続的脆弱性自動検知	common/supply_chain common/git_workflow
P2	PHPUnit 11 + pcov オフライン自動テストスイート構築	login / index / product / admin_backup	pdo_sqliteオフラインテストで実測カバレッジ 80% 以上達成	php/testing runtime_verification
P2	declare(strict_types=1) 全適用とネイティブ型・PHPStan	全6 PHPファイル	暗黙型強制バグの静的遮断と型安全カバレッジ 90%+ 達成	php/typing

推奨改善アクション計画マトリクス (P0 -> P1 -> P2 全11施策)

4. 評価指標とルール適合マトリクス

本章では、`php_legacy_shop_messy` ワークフローで実行した8つの評価ステップのスコア比較（弱点順）、全17エンジニアリングルールの適合判定マトリクス（`FAIL: 15` / `NOT_APPLICABLE: 2` / `PASS: 0`）、および重要度順に正規化された全33件の指摘事項（`CRITICAL: 10` / `HIGH: 12` / `MEDIUM: 9` / `LOW: 2`）を一覧化する。



実行監査ステップ別スコア比較 (弱点順: 全8ディメンション)

ルール適合マトリクス（全17ルール：`FAIL: 15` / `N/A: 2` / `PASS: 0`）

全評価ルールは最悪ステータス優先で集約されており、フレームワーク固有の2ルール（`php/frameworks/laravel`, `php/frameworks/wordpress`：`raw_php` のため `NOT_APPLICABLE`）を除く **適用可能な全15ルールが** `FAIL` となり、`PASS` 達成ルールは `0` 件であった。

判定	ルールID	代表ステップ	決定的エビデンス要約
FAIL	common/architecture	act-05 (AST)	0クラス・HTML+SQL混在・db.php<->helpers.php相互include循環
FAIL	common/documentation	act-02 (構造)	README.md が実質2行のみで前提・設定・テスト手順がすべて不在
FAIL	common/git_workflow	act-02 (構造)	.gitignore と .dockerignore が存在せず機密・一時ファイル混入リスク
FAIL	common/maintainability	act-05 (AST)	スーパーグローバル直参照17件・@握り潰し・die()・cURL Timeout欠如
FAIL	common/production_readiness	act-09/12	cURL/DB Timeout未設定・@無言失敗・専用/healthzとHEALTHCHECK不在
FAIL	common/runtime_verification	act-12 (動的)	Docker/php -l/起動応答通過もテスト0件で実測網羅率未計測(measured=false)
FAIL	common/security	act-02 (構造)	DB/Stripe秘密鍵平文直書き・rootコンテナ・SQLi/CMDi/LFI/unserialize
FAIL	common/security_vulnerability	act-05/14/16	DAST無認証RCE・0e突破実証 + OS CVE 12,051件 + 13 CWEシンク
FAIL	common/supply_chain	act-02/14	ロックファイル/LICENSE/SECURITY/CI/Dependabot全欠落・root Dockerfile
FAIL	common/twelve_factor	act-09 (SRE)	127.0.0.1直書き・分散getenv・ファイルSession・/tmp競合・var_dump出力
FAIL	php/code_quality	act-05 (AST)	import衛生0%・PHPDoc 0%・スパゲッティ構文・@mysqli_connect・die()
FAIL	php/dependencies	act-02 (構造)	composer.json/lock欠如(raw_php)・EOL php:7.4-apache・手動include循環
FAIL	php/modern_stack	act-02/05	PHP 7.4 EOL・extract($_POST)・md5 == '0e...'突破・TimeoutなしcURL
FAIL	php/testing	act-10 (試験)	ソース6件に対しテスト0件・phpunit.xml不在・オフライン検証分離なし
FAIL	php/typing	act-05 (AST)	全6ファイルで declare(strict_types=1) 欠落・引数/戻り値型宣言 0%
N/A	php/frameworks/laravel	act-05 (AST)	detected_frameworks=[] (Laravel/Illuminate 未使用の素PHPのため対象外)
N/A	php/frameworks/wordpress	act-05 (AST)	detected_frameworks=[] (WordPress プラグイン/テーマではないため対象外)

エンジニアリングルール適合マトリクス (FAIL -> NOT_APPLICABLE 全17ルール)

重要度順 指摘事項一覧（全33件：`CRITICAL: 10` / `HIGH: 12` / `MEDIUM: 9` / `LOW: 2`）

各監査ステップ（SAST・Trivy・ライブ DAST `web_pentest`）から抽出・正規化された全33件の指摘事項を `CRITICAL -> HIGH -> MEDIUM -> LOW` の厳密な順序で3分割して以下に示す（機密リテラル値はすべて `[REDACTED]` 処理済み）。

重要度	ルールID	該当箇所	検出事象・CWE/CVE/DAST根拠	推奨修正策
CRIT	security_vulnerability	Dockerfile:1	php:7.4-apache OS CVE 12,051件 (CRIT 62)	php:8.3-apache-bookwormへ刷新
CRIT	common/security	includes/db.php:6	DB_PASSWORDとStripe鍵の平文直書き(CWE-798)	既存鍵を即時失効し環境変数注入化
CRIT	security_vulnerability	admin_backup.php:6	SAST: 無認証 shell_exec('tar ...'.$target) (CWE-78)	認可+CSRF強制・シェル呼出を廃止
CRIT	security_vulnerability	admin_backup.php:6	DAST実証: POSTで PNI_RCE_CANARY_8842 実行(CWE-	即時アクセス遮断・ZipArchive置換
CRIT	security_vulnerability	includes/db.php:7	Trivy Secret検出: stripe-secret-token [REDACTED]	Stripe鍵ローテーション・Secret管理
CRIT	security_vulnerability	index.php:4	$_COOKIE['shop_cart'] の危険な unserialize(CWE-50	Server Sessionまたは署名付JSON化
CRIT	security_vulnerability	index.php:8	index/product/login 3箇所の生SQL連結(CWE-89)	PDOプリペアドステートメントへ統一
CRIT	security_vulnerability	index.php:25	include $_GET['page'].'.php' によるLFI(CWE-98)	動的include廃止・固定ルーティング化
CRIT	security_vulnerability	login.php:17	DAST実証: '240610708' で管理者ログイン突破(CWE-69	0eフォールバック撤去・Fail-Closed化
CRIT	php/modern_stack	login.php:17	SAST: extract + md5 + == '0e...' 型強制(CWE-697)	extract削除・password_verify()移行

重要度順 検出指摘一覧 (1/3: CRITICAL 全10件 — DASTライブ実証2件を含む)

重要度	ルールID	該当箇所	検出事象・CWE/DAST根拠	推奨修正策
HIGH	production_readiness	helpers.php:7	cURLにCURLOPT_TIMEOUT未設定でハング危険	CONNECTTIMEOUT=3/TIMEOUT=10設定
HIGH	common/supply_chain	Dockerfile:1	EOL PHP 7.4・USER未指定・.dockerignore不在	ダイジェスト固定・非root・ignore追加
HIGH	security_vulnerability	Dockerfile:1	Trivy IaC DS-0002: root権限でApache稼働	USER www-data による最小権限実行
HIGH	security_vulnerability	admin_backup.php:1	全4公開エンドポイントに認可・CSRF・型検証なし	public/集約・認可ミドルウェア強制
HIGH	security_vulnerability	helpers.php:9	CURLOPT_SSL_VERIFYPEER=false & URLトークン	TLS検証有効化・Bearerヘッダ送信
HIGH	security_vulnerability	index.php:21	SAST: index/product/login 3箇所の生echo(CWE-79)	Twig/Bladeまたはhtmlspecialchars徹底
HIGH	security_vulnerability	index.php:21	DAST実証: <pni-xss-canary-7f3a> 生HTML反射	検索語出力のUTF-8 HTMLエスケープ強制
HIGH	security_vulnerability	index.php:25	DAST実証: ../../../../etc/passwd でLFIエラー露出	動的include撤廃・display_errors=Off
HIGH	security_vulnerability	product.php:14	DAST実証: promo_html=<pni-promo-xss-9a2b> 反射	生HTMLクエリ廃止・テンプレートID化
HIGH	php/dependencies	Dockerfile:1	composer.json/lock未導入・PHP 7.4 EOL指定	Composer PSR-4導入・PHP 8.3移行
HIGH	runtime_verification	index.php:1	php -l通過もテスト0件で実測網羅率未計測	PHPUnit 11 + pcovで網羅率80%+計測
HIGH	php/testing	login.php:1	本番6ファイルに対し自動テスト・phpunit.xml不在	pdo_sqliteオフライン単体テスト整備

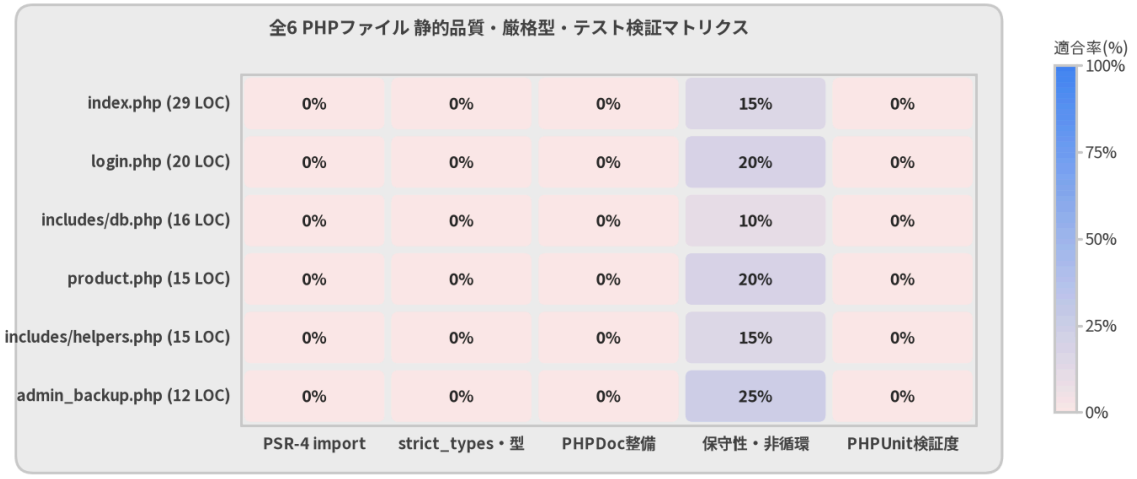
重要度順 検出指摘一覧 (2/3: HIGH 全12件 — DASTライブ実証LFI/XSSを含む)

重要度	ルールID	該当箇所	検出事象・根拠	推奨修正策
MED	common/architecture	includes/db.php:2	db.php <-> helpers.php 相互include循環・HTML+SQL混	PSR-4単方向レイヤ(Controller/Service/Repo)化
MED	common/twelve_factor	includes/db.php:4	DB_HOST='127.0.0.1'直書きと分散getenv()呼び出し	型付きAppConfigクラスで環境変数を一元読込
MED	production_readiness	Dockerfile:5	専用 /healthz・/readyz ルートおよび HEALTHCHECK/Re	healthz・/readyz JSON実装とHEALTHCHECK定義
MED	common/twelve_factor	helpers.php:12	?debug=1でのvar_dump出力と固定/tmp/shop_backup/	var_dump撤廃・PSR-3 JSONログ・外部保存化
MED	php/code_quality	includes/db.php:12	@mysqli_connect握り潰し・global変数・die(mysqli_er	@・global・die廃止とPDO例外ハンドラ統一
MED	common/supply_chain	README.md:1	LICENSE・SECURITY.md・Dependabot・CIワークフロー	統治文書追加と最小権限GitHub Actions整備
MED	security_vulnerability	Dockerfile:1	DAST実証: X-Frame-Options/nosniff欠落・TRACE 200応	ServerTokens Prod・ServerSignature Off・expose_php=Off・CSP/nosniff付与
MED	common/supply_chain	Dockerfile:4	COPY . /var/www/html/ による全ファイル露出・root実	.dockerignore追加・public/のみ公開・非root化
MED	php/typing	helpers.php:4	全6ファイル strict_types=1 欠落・全2関数で型宣言なし	declare(strict_types=1)全適用・ネイティブ型・PHPStan
LOW	common/documentation	README.md:1	見出し1つと1文のみで環境変数・起動・テスト手順が欠	Standard Readme構成で構築・検証手順を明記
LOW	common/git_workflow	(repo root)	.gitignore不在により機密ファイルや一時成果物の混入危	vendor/・.env等を除外する.gitignoreを追加

重要度順 検出指摘一覧 (3/3: MEDIUM 9件・LOW 2件)

5. コード品質・型安全性・ランタイム検証詳細

本章では、PHP AST 静的解析から得られたコード品質・型安全性指標（`import_hygiene=0.0%`, `type_safety=0.0%`, `docstring=0.0%`）の内訳と、隔離 Docker コンテナ上で実行されたビルド・`php -l` 構文検査・自動テスト・`php -S` 起動スモーク検証（RV-01〜RV-04）の決定的結果を詳細に分析する。



モジュール × 品質ディメンション適合度マトリクス (0 - 100%)

静的コード品質・厳格型安全性・保守性の欠陥詳細

- `declare(strict_types=1)` **カバレッジ 0% とネイティブ型不在 (**`php/typing`**)**: 全6ファイル（`index.php:1`, `product.php:1`, `login.php:1`, `admin_backup.php:1`, `includes/db.php:1`, `includes/helpers.php:1`）で `declare(strict_types=1);` が宣言されておらず、PHP の暗黙スカラー型変換が全面的に有効となっている。また定義されている2つの関数 `get_db_conn()`（`includes/db.php:9`）と `fetch_partner_catalog($endpoint)`（`includes/helpers.php:4`）にも引数型・戻り値型および `/** ... */` PHPDoc が一切付与されていない（`type_safety_coverage_pct=0.0%`, `docstring_coverage_pct=0.0%`）。

- **スーパーグローバル直参照16箇所＋`global`変数結合（`php/code_quality`, `common/maintainability`）**: リクエスト抽象化層が存在せず、`$_GET`（8箇所）・`$_POST`（5箇所）・`$_COOKIE`（2箇所）・`$_SERVER`（1箇所）・`$_SESSION`に加え、`includes/db.php:11`の`global $LAST_DB_ERR;`を含む計**17箇所の可変グローバル状態結合**が確認された。特に`login.php:6`の`extract($_POST);`は任意のPOSTキーでローカルシンボルテーブルを上書きできる危険なレガシーイディオムである。
- **`@mysqli_connect`エラー握り潰し・`die()`・cURLタイムアウト欠如（`common/production_readiness`）**: `includes/db.php:12`は`@mysqli_connect(...)`の`@`演算子で接続失敗の原因（認証エラー・DNS失敗・ソケット枯渇）を完全に握り潰す一方、`product.php:9`は`die("Database query failed: " . mysqli_error($conn));`で内部SQLエラーメッセージをそのままクライアントへ返してプロセスを強制終了する。さらに`includes/helpers.php:7-13`は`CURLOPT_CONNECTTIMEOUT`/`CURLOPT_TIMEOUT`未設定のまま外部通信を行い、`$_GET['debug']`指定時に生レスポンスを`var_dump($raw)`で出力している。

コンテナ動的検証（`php -l`構文通過・テスト0件・`php -S`起動スモーク結果）

隔離Docker環境における動的検証（`run_check`）では、同梱`Dockerfile`ビルド（`1.11s`）、PHP 8.3ハーネスビルド（`0.94s`）、PHP 8.3.36構文検査（`php -l`、`1.01s`）、および`php -S`組込みサーバ起動スモーク応答（`smoke_check: status=passed`、`/up -> 200`フォールバック応答、`1.06s`）は通過したが、PHPUnitテストが`0`件のため実測カバレッジは完全に未計測（`measured=False`）であり、専用の`/healthz`ルートや`Dockerfile`の`HEALTHCHECK`命令も欠如している。

PASSED (1.11s)

同梱 Dockerfile ビルド

php:7.4-apache + mysqli

PASSED (1.01s)

PHP 8.3 構文検査 (php -l)

Harness 0.94s / 6件構文OK

0 TESTS (N/A)

オフライン PHPUnit

自動テスト0件・pcov未計測

PASSED* (1.06s)

Web 起動スモーク応答

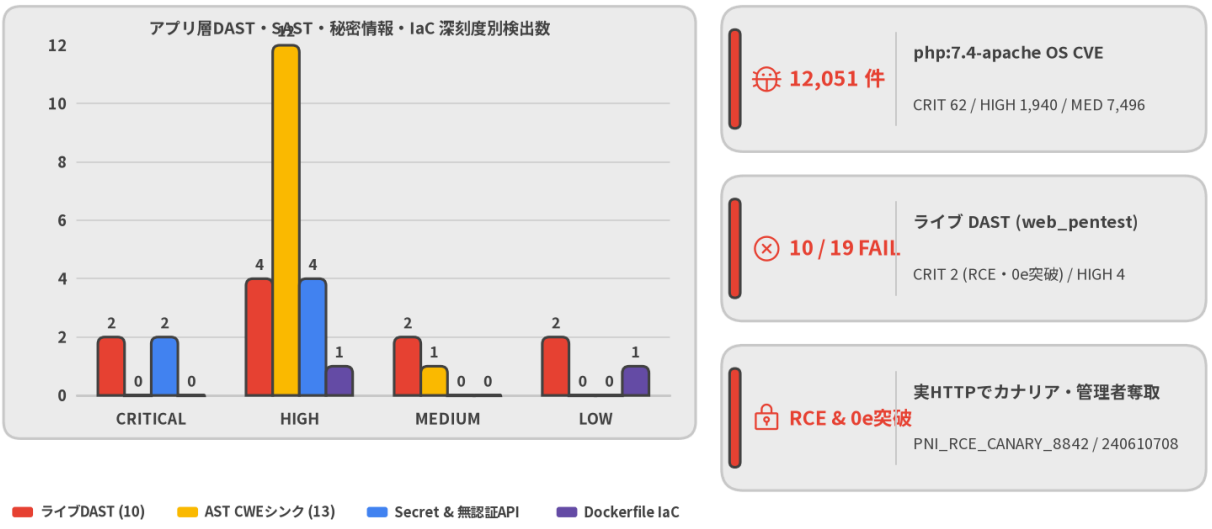
/up -> 200 (index.php fallback)

検証ゲート	ステータス	所要時間	決定的実行結果・ログエビデンス
RV-01a: 同梱 Dockerfile	PASSED (exit 0)	1.11s	FROM php:7.4-apache 上で docker-php-ext-install mysqli が成功。ただし EOL ランタイム・root 実行・OS CVE 12,051件を内包
RV-01b: PHP 8.3 ハーネスビルド	PASSED (exit 0)	0.94s	php/raw.dockerfile (PHP_VERSION=8.3) のクリーンイメージビルドがエラーなく完了
RV-02: オフライン構文・単体試験	PASSED* (0 tests)	1.01s	--network=none 下で全6ファイルの php -l (PHP 8.3.36) 構文検査は通過したが、tests/ 不在のため実行された PHPUnit テストは 0 件
RV-03: pcov 実測カバレッジ	UNMEASURED (measured=false)	0.00s	PHPUnit テストが 0 件のため Clover カバレッジは未生成 (line=None%, branch=None%, 0/0 statements) であり回帰防止能力は皆無
RV-04: Web API スモーク起動	PASSED* (HTTP 200)	1.06s	php -S (127.0.0.1:18080) が /up -> 200 を返したが index.php HTML へのフォールバック応答であり、専用 /healthz と HEALTHCHECK は不在

コンテナ動的検証KPIと RV-01〜RV-04 ゲート別実行結果内訳

6. セキュリティ・脆弱性・サプライチェーン分析

本章では、Trivy (`0.69.3`) によるコンテナイメージ・ファイルシステム検査（ `security_scan`)、PHP AST によるハードコード秘密情報・13件の CWE 危険シンク・無認証 HTTP エンドポイント境界解析、および隔離ループバックコンテナ上で `.pni/pentests/examples/php/legacy_shop_messy/` 全19プローブを実行した動的 Web ペネトレーションテスト（ `web_pentest` / SEC-06）の評価結果を統合して詳述する。



ライブDAST (10 FAIL)・AST CWE危険シンク・秘密情報・IaC不備およびOS層CVE (12,051件) 内訳

セキュリティゲート（SEC-01〜SEC-05）およびサプライチェーンの詳細評価

- **サプライチェーン欠落と EOL `php:7.4-apache` コンテナの `12,051` OS 層 CVE (`SEC-01` / `SEC-02`)**: `composer.json` / `composer.lock` が存在しないためアプリ依存パッケージ CVE は `0` 件だが、`Dockerfile:1` の `FROM php:7.4-apache`（2022年11月 EOL・Debian 11.5 凍結状態）に対する Trivy イメージスキャンにより、**計 `12,051` 件（ `CRITICAL: 62` , `HIGH: 1,940` , `MEDIUM: 7,496` , `LOW: 2,296` ）** の OS パッケージ脆弱性が検出された。

代表例として `apache2@2.4.54-1~deb11u1` の HTTP リクエストスプリッティング/mod_rewrite 脆弱性（`CVE-2023-25690`, `CVE-2024-38474`, `CVE-2024-38476`、いずれも CVSS 9.8）や `curl` / `libcurl4@7.74.0` の SOCKS5 ヒープオーバーフロー（`CVE-2023-38545`、CVSS 9.8）が含まれる。さらに Trivy IaC で `DS-0002`（`HIGH`：`root` 実行）と `DS-0026`（`LOW`：`HEALTHCHECK` 不在）が検出された。

- **ハードコードされた DB パスワード・Stripe Live 秘密鍵 (`SEC-03` / CWE-798)**: `includes/db.php:6` に本番 MySQL パスワード定数（`DB_PASSWORD = '[REDACTED]'`）、同 `7` 行目に Stripe Live API シークレットキー（`$stripe_api_secret = 'sk_live_[REDACTED]'`、Trivy Secret Scanner が `CRITICAL` `stripe-secret-token` として検出）が平文で埋め込まれており、コンテナ内の `/var/www/html/includes/db.php` にそのままコピーされている。
- **CWE 危険コードシンク 13件と無認証管理エンドポイント (`SEC-04` / `SEC-05`)**:
  - **CWE-78 OS コマンドインジェクション (`admin_backup.php:6`)**: セッション確認（`$_SESSION['is_admin']`）も CSRF 対策もない公開フォームから `$_POST['backup_dir']` を `shell_exec("tar -czf /tmp/shop_backup.tar.gz " . $target)` へ直接連結しており、未認証のまま任意 OS コマンドを実行できる。
  - **CWE-98 LFI (`index.php:25`) & CWE-502 `unserialize` (`index.php:4`)**: `include $_GET['page'] . '.php';` による任意 PHP ファイル実行・パストラバーサル、および改ざん可能な `$_COOKIE['shop_cart']` の `unserialize()`（`allowed_classes` 制限なし）による PHP Object Injection が存在する。
  - **CWE-89 SQLi（3箇所）& CWE-79 反射型 XSS（3箇所）**: `index.php:8`（`$_GET['q']`）、`product.php:6-7`（`$_GET['cat']`）、`login.php:12`（`$_POST['username']`）がすべて文字列連結 SQL を `mysqli_query()` で実行し、`index.php:21`、`product.php:14`（`$_GET['promo_html']`）、`login.php:19` が未エスケープのまま HTML 出力している。
  - **CWE-621 / CWE-327 / CWE-697 マジックハッシュ認証バイパス (`login.php:6-17`)**: `extract($_POST)` による変数汚染、ソルトなし `md5()`、および初期値 `$expected_hash = '0e462097431906509019562988736854'` と緩い比較 `$pw_hash == $expected_hash` の組み合わせにより、`'240610708'` 等のマジックハッシュ入力で `0 == 0` が成立し管理者セッションを奪取できる。
  - **CWE-295 TLS 証明書検証無効化 (`includes/helpers.php:9`)**: `CURLOPT_SSL_VERIFYPEER => false` により HTTPS 証明書検証を無効化した上で、クエリパラメータ `?token=` にトークンを平文付与している。

動的Webペネトレーションテスト (`web_pentest` / `.pni/pentests/`, SEC-06)

隔離ループバック環境（`127.0.0.1`）に対して `.pni/pentests/examples/php/legacy_shop_messy/`（`test_baseline_http.py`, `test_target_surface.py`, `test_app_probes.py`）の計 `19` プローブを実行した結果、`status = "failed"`（`9` **PASS** / `10` **FAIL**、`CRITICAL: 2` • `HIGH: 4` • `MEDIUM: 2` • `LOW: 2`、**実行時間** `4.31s`）となり、SAST で検出された高リスク経路がライブ HTTP リクエスト単発で実際に成立することが決定的に証明された。特に無認証 `POST /admin_backup.php` におけるシェル注入カナリア `PNI_RCE_CANARY_8842` の実行出力（`CWE-78` CRITICAL）および `POST /login.php` における `'240610708'` マジックハッシュ送信での `'Logged in as admin'` 奪取（`CWE-697` CRITICAL）は、外部公開と同時に即座に侵害されることを示している。

判定 / 深刻度	プローブテスト / CWE	対象エンドポイント	ライブHTTP応答エビデンス	修正・ゲート化方針
FAIL [CRITICAL]	test_app_probes.py CWE-78 (OS CMDi)	POST /admin_backup.php (admin_backup.php:6)	未認証POST backup_dir で注入した echo カナリア PNI_RCE_CANARY_8842 出力	shell_exec撤廃・Admin認可+CSRF・Symfony Process配列引数Job化
FAIL [CRITICAL]	test_app_probes.py CWE-697 (Auth突破)	POST /login.php (login.php:17)	password='240610708' 送信で 0==0 成立 HTTP 200 'Logged in as admin' 応答	0e定数と==比較廃止・Fail-Closed・password_verify(bcrypt/Argon2)化
FAIL [HIGH]	test_target_surface.py CWE-306 (無認証操作)	POST /admin_backup.php (admin_backup.php:4)	セッションCookieなしの POST が 401/403 拒否されず HTTP 200 応答	管理系ルートへ認証ミドルウェアおよびCSRFトークン検証を強制
FAIL [HIGH]	test_target_surface.py CWE-22 (LFI露出)	GET /index.php (index.php:25)	page=../../../../etc/passwd 指定で HTTP 200 "Failed opening '" 露出	動的include撤廃・固定ルートマップおよび display_errors=Off 徹底
FAIL [HIGH x2]	test_target_surface & app_probes (CWE-79)	GET /index.php & /product (index:21, product:14)	<pni-xss-canary-7f3a> および promo_html =<pni-promo-xss-9a2b> が生HTML反射	Blade/Twig自動エスケープ導入と promo_html生HTMLクエリ仕様の廃止
FAIL [MED x2]	test_baseline_http.py CWE-1021 / CWE-693	GET / & TRACE / (Dockerfile / Web層)	HTML応答にX-Frame-Options/CSP欠落 および TRACE / -> HTTP 200 応答	X-Frame-Options: DENY / CSP付与・本番SAPIでTRACEメソッド405遮断
FAIL [LOW x2]	test_baseline_http.py CWE-200 / CWE-693	GET / (Dockerfile / Web層)	X-Powered-By: PHP/8.3.36 露出 X-Content-Type-Options=<absent>	expose_php=Off 設定と全応答への X-Content-Type-Options: nosniff付与
PASS (9件) [基準確認]	baseline & surface (.env / .git / 404等)	GET /.env, /.git/HEAD, composer.json 他	機密ドットファイル非露出・異常長入力での500クラッシュなし (9/19 PASS)	移行後も全19プローブ 100% PASS を CI リリースゲートとして継続監視

動的Webペネトレーションテスト (.pni/pentests/ 全19プローブ: 9 PASS / 10 FAIL) ライブ実証結果

ゲート / 深刻度	CWE / CVE / ID	対象箇所	脆弱性メカニズム・攻撃影響	必須緩和・修正策
SEC-04 / 05 / 06 [CRITICAL]	CWE-78 / CWE-306 Command Inj.	admin_backup .php:4-6	無認証POST backup_dir を shell_exec へ直結しDASTでもRCEカナリア実証	認可+CSRF必須化・シェル呼出廃止 (ZipArchive / 固定ジョブ化)
SEC-04 / 06 [CRITICAL]	CWE-98 / CWE-502 LFI & Unserialize	index.php:4, index.php:25	Cookie unserialize() と動的 include $_GET['page'].'.php' (DAST LFI実証)	json_decode + HMAC署名へ変更・ matchホワイトリストルーティング
SEC-04 [CRITICAL]	CWE-89 (x3) SQL Injection	index:8 / login:12 product.php:6-7	3画面で未エスケープ入力をSELECT文 へ直接連結し全件抽出・認証突破	PDO (EMULATE_PREPARES=false) プリペアドステートメント全面適用
SEC-04 / 06 [CRITICAL]	CWE-697 / 327 / 621 Auth Bypass	login.php:6-17	extract + md5 + 緩い == '0e4620...' ('240610708' でDAST管理者奪取実証)	extract削除・password_hash(Argon2) + password_verify・Fail-Closed化
SEC-03 [CRITICAL]	CWE-798 / Trivy stripe-secret-token	includes/db.php :6-7	DB_PASSWORDとStripe本番秘密鍵 ([REDACTED])をソースに平文埋込	両クレデンシャル即時ローテーション ・Secret Manager / 環境変数注入
SEC-01 [CRITICAL]	CVE-2023-25690 CVE-2023-38545他	Dockerfile:1 (php:7.4-apache)	EOL Debian 11.5層に OS CVE 12,051件 (CRIT 62 / HIGH 1,940, CVSS 9.8)	php:8.3-apache-bookworm へ刷新し ダイジェスト固定・定期再ビルド
SEC-04 / 06 [HIGH]	CWE-79 (x3) Reflected XSS	index:21 / login:19 product.php:14	q・promo_html・username を生echo (DASTでカナリアHTMLタグ反射実証)	Twig/Blade自動エスケープまたは htmlspecialchars(ENT_QUOTES)徹底
SEC-04 [HIGH]	CWE-295 Insecure TLS	includes/helpers .php:7-9	CURLOPT_SSL_VERIFYPEER=false かつ URLクエリにトークン付与(MITM漏洩)	VERIFYPEER=true・VERIFYHOST=2・ Authorizationヘッダ・Timeout設定
SEC-02 / 06 [HIGH / MED]	Trivy DS-0002 / CWE-1021 / 693	Dockerfile:1-5	root稼働・HEALTHCHECK不在に加え CSP/XFO/nosniff欠落・TRACE有効	USER www-data・HEALTHCHECK定義 ・セキュリティヘッダー括付与

SEC-01〜SEC-05 セキュリティゲート・主要CVE・CWE危険シンク詳細一覧